Lecture-5

Software Design

Goals of Design

- Decompose system into components
  - i.e. identify the software architecture
- Describe component functionality
  - informally or formally
- Determine relationships between components
  - identify component dependencies
  - determine inter-component communication mechanisms
- Specify component interfaces
  - Interfaces should be well defined
    - facilitates component testing and team communication

```
➢ Intr-o app de mari dimensiuni -> ne trebuie mecanisme de gestionare a
complexitatii unui sistem
➢ De ex, am folosit CRC cards si am gasit toate clasele din sistem -> nr de
clase e 1500 -> cum le gestionam? (astea sunt rezultate doar din modelare)
➢ Pt asta avem nevoie de proiectarea SW-ului -> SW design (design in sensul
de proiectare)
 ➢ Ca sa avem o ordine intre aceste dependente si ca sa specificam clar ce
poate sa foloseasca o componenta dintr-o alta component folosim INTERFATA ->
ea trebuie sa fie ft bine delimitate ca sa delimitam ft bine interactiunile
dintre componente
 ➢ O clasa e o component? Da , clasa in sine e un elem de proiectare -> e o
component -> dar clasa e o forma de abstarctizare, dar nu e suficienta
```

1. **Decompose system into components**

- În această etapă stabilim **arhitectura generală a sistemului**.
- Ne întrebăm: din ce părți va fi compus sistemul? Care sunt modulele majore?
- Fiecare componentă va fi responsabilă pentru o anumită zonă de funcționalitate.
- La acest nivel, nu discutăm încă de clase concrete — ci de module, subsisteme, servicii.

2. Describe component functionality

- Ce face fiecare componentă? Care este responsabilitatea ei?
- Putem descrie funcționalitatea:

    - **Informal:** descrieri textuale, simple.
    - **Formal:** prin modele UML, diagrame de secvență, cazuri de utilizare etc.

3. Determine relationships between components

- Trebuie să definim cum colaborează aceste componente între ele.
- Identificăm:
    - **Dependințe**: care componente depind una de alta?
    - **Mecanisme de comunicare**: cum fac schimb de informații?
        - apel direct (funcție/metodă),
        - mesaje (event bus),
        - API-uri REST,
        - mesaje asincrone (queue, pub/sub).

4. **Specify component interfaces**

- Fiecare componentă trebuie să expună o **interfață clar definită**:
    - ce date primește,
    - ce date returnează,
    - ce metode oferă.
- O interfață bine definită
    - permite testarea independentă a componentelor,
    - permite ca echipele să lucreze în paralel pe componente diferite,
    - reduce riscul de blocaje în dezvoltare.

➢ Elem cheie de aici e ideea de descompunere -> mai exact component care rezulta in urma descompunerii

Decomposition

WHY ?

Handle complexity by splitting large problems into smaller problems, i.e. "*divide and conquer*" methodology

1. Select a piece of the problem
   - initially, the whole problem
2. Determine the components in this piece using a design paradigm
   - e.g. functional, structured, object-oriented, generic, etc.
3. Describe the components interactions
4. Repeat steps 1 through 3 until some termination criteria is met
   - e.g., customer is satisfied, run out of money, etc. ;-)

```
➢ Pornim de la o problema mare si prin metoda “divide and conquer” ,
descoperim componentele de la nivelul de la care facem descompunerea, scriem
interactiunile dintre componente si repetam ciclul asta pt ajungem la un
nivel de rafinare destul de bun
```

A component is ...

- ... a software entity encapsulating the representation of an abstractio
- ... a vehicle for hiding at least one design decision
- ... a "work" assignment
  - for a programmer or group of programmers
- ... a unit of code that
  - has one (or more) name(s)
  - has identifiable boundaries
  - can be (re-)used by other components
  - encapsulates data
  - hides unnecessary details
  - can be separately compiled

```
➢ Ce e o componenta? Se mai poate spune ca sunt entitati, subsisteme etc. -
> ele ascund un volum de complexitate functionala si interactioneaza cu alte
componente -> o alta descriere ar fi ca o componenta e o entitate SW care
incapsuleaza o abstractiune
➢ Abstractizarea e un fel de a modela
```

```
➢ O componenta poate fi si un set bine delimitat de task-uri -> un grup de
programatori -> un SCRUM team
```

What is good Design?

- The temptation of "correct design"
  - insurance against "design catastrophes"
  - design methods that guarantee the "correct design"

> *A good design is one that balances trade-offs to minimize the total cost of the system over its entire lifetime*
> *[...]*
> *a matter of avoiding those characteristics that lead to bad consequences.*
> **Coad & Yourdon**

There is no correct design! You must decide!

- Need of criteria for evaluating a design
- Need of principles and rules for creating good designs

```
➢ Putem face un system sw care pare perfect, dar de fapt sa nu fie asa->
cum ne dam seama? In timp
```

Modularity

- A modular system is one that's structured into identifiable abstractions called components
  - Components should possess well-specified abstract interfaces
  - Components should have high cohesion and low coupling

> *A software construction method is modular*
> *if it helps designers produce software systems*
> *made of autonomous elements*
> *connected by a coherent, simple structure.*
> **B. Meyer**

low coupling = putine legaturi cu alte componente, ca sa nu depinda prea multe intre ele

```
➢ Un sistem e modular atunci cand el este format din componente
autonome(libere,independente) conectate intre ele printr-o structura
```

```
coerenta si simpla (relatiile sunt dependente)
➢ Caracteristici principale : autonomia sa fie cat mai mare (coeziunea unui
element sa fie cat mai mare-> cat de bine definit e el) -> de ce sa fie mai
mare? Pt ca dependetele cu celelalte module sa fie mai putine, mai simple
```

Modular system

> *Un sistem modular este structurat în componente bine definite.*

- Adică sistemul este împărțit în bucăți clare, fiecare componentă având o responsabilitate bine definită.
- Aceste componente sunt numite **abstracții identificabile** — le putem izola, înțelege, testa și modifica independent.

Două caracteristici esențiale ale componentelor

1️⃣ Well-specified abstract interfaces

> Interfețe abstracte bine specificate

- Fiecare componentă trebuie să aibă o interfață clară:
  - Ce date primește?
  - Ce date returnează?
  - Ce metode oferă?
- Interfața trebuie să ascundă detaliile interne (encapsulation).
- Exemple:
  - API public bine definit.
  - Interfață Java (interface), sau protocol în alte limbaje.

Beneficii:

- Poți schimba implementarea internă fără să afectezi restul sistemului (atâta timp cât interfața rămâne constantă).
- Permite echipelor să lucreze în paralel.

2️⃣ High cohesion and low coupling

Aici e cheia designului bun. Să detaliem:

🔵 High Cohesion (Coeziune mare)

> *O componentă este bine "lipită în interior"*

- Toate elementele dintr-o componentă sunt direct legate de scopul principal al acelei componente.
- Metodele și datele au legătură logică strânsă între ele.
- Fără funcționalități amestecate care nu au legătură.

Exemplu bun (high cohesion):

Componenta `Payment`:

- gestiune plăți,
- procesare carduri,
- validare tranzacții.

Exemplu rău (low cohesion):

Componenta `MiscellaneousUtils`:

- face validare email,
- trimite SMS,
- loghează erori,
- calculează TVA.

Regula simplă:
👉 O componentă trebuie să aibă un singur scop bine definit.

🔵 Low Coupling (Cuplaj mic)

> *Componentele sunt cât mai independente unele de altele*

- Componentele colaborează prin interfețe, dar evităm să fie puternic dependente una de alta.
- Dacă modifici o componentă, să nu trebuiască să modifici și multe altele.
- Relațiile între componente să fie minime și clare.

Exemplu bun (low coupling):

- `OrderProcessing` comunică cu `Inventory` doar printr-un API: `checkAvailability()`.
- Nu cunoaște detalii interne ale `Inventory`.

Exemplu rău (high coupling):

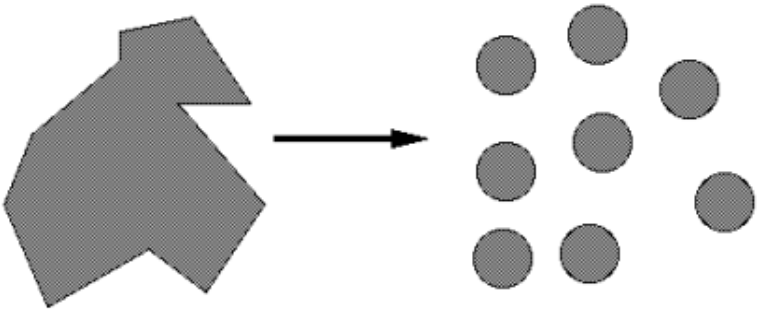
- `OrderProcessing` accesează direct baza de date a `Inventory` și modifică direct tabelele acestuia.

Meyer's Five Criteria for Evaluating Modularity

- **Decomposability**
  - Are larger components decomposed into smaller components?
- **Composability**
  - Are larger components composed from smaller components?
- **Understandability**
  - Are components separately understandable?
- **Continuity**
  - Do small changes to the specification affect a localized and limited number of components?
- **Protection**
  - Are the effects of run-time abnormalities confined to a small number of related components?

1. Decomposability

- Decompose problem into smaller sub-problems that can be solved separately
  - **Goal**: Division of Labor
    - keep dependencies **explicit** and **minimal**
  - Example: Top-Down Design
  - Counter-example: Initialization Module
    - initialize everything for everybody



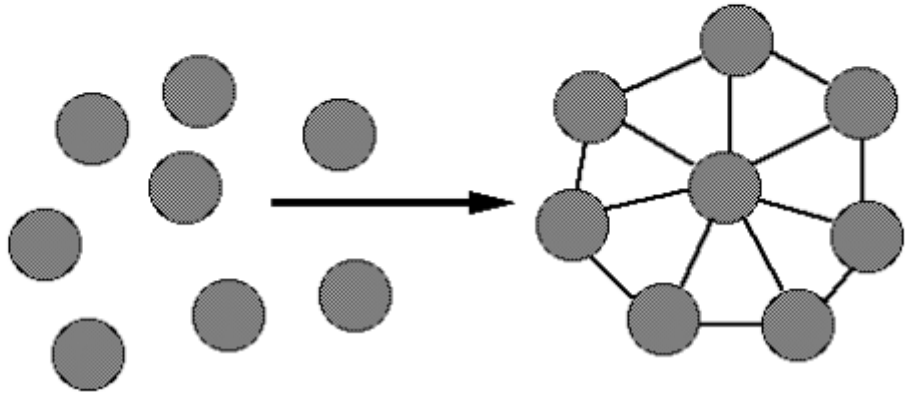
Practic descompunem o problema in mai multe probleme mai mici pe care le rezolvam independent. Dupa rezolvarea acestora, se unesc si trebuie sa dea toata problema rezolvata

```
➢ Un sistem e decompozabil? Da -> indifferent la ce nivel sunt, daca pot sa
descompun component in componente mai mici -> sistemul e decompozabil -> nu
e ceva binary, avem mai multe nivele, pe un anumit nivel poate fi
decompozabil, iar pe nivelele de mai jos nu si invers
    o ex bun: programare TOPDOWN -> am un input, il citesc, il scriu la
output
    o un contraexemplu ar fi atunci cand undeva exista un modul de
initializare (la un counter) : mereu cand avem ceva care face de toate pt
toti avem decompozabilitate (cand ceva face prea multe) -> pt ca nu mai avem
ierarhia de descompunere
```

Ex curs : mobila de la ikea, ai niste piese mici, ai instructiuni, urmaresti planul ca sa le unesti si ajungi la un singur output care este mobila

2. Composability

- Freely combine modules to produce new systems
  - **Reusability** in different environments → components
  - Example: Math libraries; UNIX command & pipes



Aici avem mai multe bucati mici pe care le putem combina in mai multe moduri, asa cum vrem noi

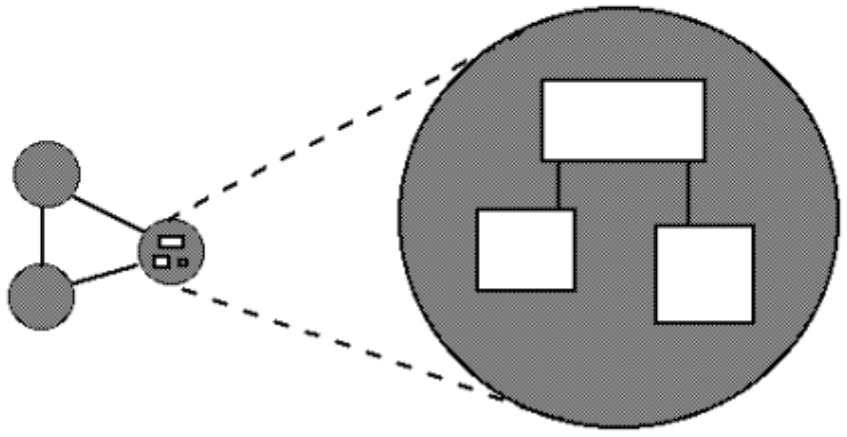
Ex curs : Lego urile vechi, nu primesti instructiuni. Primeai doar o galeata cu piese si tu faceai ce voiai

```
➢ Un sistem e compozabil? Da -> componentele mai mari se formeaza din
componente mai mici? Nu neaparat -> se dau niste componente si din ele se
pot construi mai multe sisteme
    o Contraexemplu : ai mai multe piese pt a construi un dulap -> daca le
```

```
iei pas cu pas, va duce la produsul final, un dulap -> piesele respective
(care sunt pt dulap) vor construi doar un dulap -> e modular, dar respectand
regula de compozabilitate -> mobila a fost gandita modular ca sa se poata
descompune (folosind criteriul decompozabilitatii) ca mai apoi tu sa o poti
compune (criteriul compozabilitatii) -> dar e decompozabilitate -> de ce nu
e compozabilitate ? pt ca din piesele astea pot face doar un lucru, nu mai
multe -> ca sa poata fi compozabilitate, trebuie sa avem piese care sa poata
sa faca mai multe lucruri o Un exemplu : ar fi piesele de lego, dar nu
intotdeauan -> cand de ex avem turnul Eiffel construit din piese de lego, il
descompui in piese mai mici care sunt specifice(nu se pot refolosi pt
altceva), NU GENERICE , dar cand il recompui tot turnul Eiffel ti-l vei
construi -> ceea ce duce la criteriul de decompozabilitate -> in schimb,
daca folosesc niste piese de lego clasice, GENERICE , pot construi orice cu
ele (dim mai mici sau mai mari) -> doar in cazul asta avem compozabilitate
    o Compozabilitatea e un super obiectiv : daca fac decompozabilitate in
loc de compozabilitate, adica daca am un sistem modular care e proiectat in
asa fel incat sa “break down in multiple levels” , adica sa pot vedea
sistemul ala descompus succesiv in mai multe piese -> atunci sistemul meu e
in continuare modular -> nu e neaparat sa fie compozabil ca sa fie modular ,
e sufficient sa fie decompozabil ca sa fie modular
```

3. Understandability

- Individual modules understandable by human reader
  - Counter-example: Sequential Dependencies (A | B | C)
    - contextual significance of modules



-> Sa scriem cod care sa fie inteles de catre oameni

```
➢ Un sistem e usor de inteles? Aici e clar vorba de autonomia modulului ->
pot sa inteleg un modul in izolare? Ideea e sa fie modulul sufficient de
```

```
autonomy ca sa il pot intelege in izolare -- ->
    o Ex: o functie calculeaza radical , apoi vreau sa calculez log din ceva
, fac o functie noua? Nu -> fac o functie numita calculeaza care primeste un
flag (ca un enum, o valoare) -> si atunci, daca val aia e radical -> face x
lucru , daca val aia e de ex 7 atunci face logaritmul -> tu nu stii exact ce
va face functia aia -> sensul unui modul e dat de cine il utilizeaza -> se
numesc dependente secventiale -> daca il apelez pe b dupa a , va face
altceva decat daca l-as apela pe c inainte de b sau altceva daca l-as apela
pe b si apoi pe a -> sensul modului b nu e independent de context -> noi
vrem sa construim module cat mai independente de context
```

🔴 Contra-exemplu: Sequential Dependencies (A → B → C)

- Aici avem module care depind strict unul de altul într-o anumită ordine.
- Nu poți înțelege modulul B fără să știi tot ce face A.
- Nu poți înțelege C fără să știi exact cum funcționează B.

Problema:

- Apare o **dependență secvențială** (chained dependency).
- Dacă vrei să modifici C, s-ar putea să trebuiască să înțelegi toată seria A → B → C.
- Devine greu de întreținut și de extins.

🔵 Ce înseamnă contextual significance of modules?

- În design bun, modulele trebuie să aibă **semnificație proprie**, clară, indiferent de context:
  - Să știi clar ce responsabilitate are modulul doar uitându-te la el.
  - Să nu fie nevoie să analizezi toate celelalte module dependente ca să înțelegi scopul și funcționarea lui.

Exemplu bun:

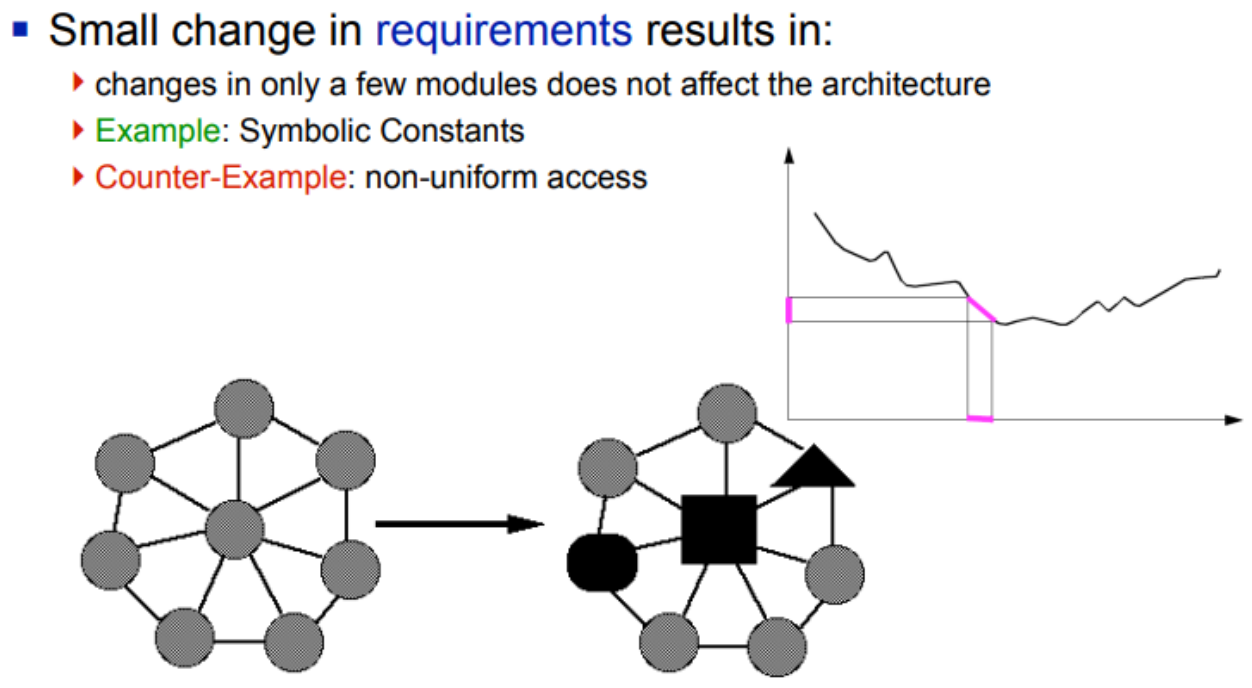
- Modul `UserManagement` gestionează utilizatori: creează, șterge, modifică.
- Poți înțelege acest modul fără să trebuiască să știi cum funcționează `OrderProcessing`.

🔵 Desenul din slide:

- Imaginea arată două niveluri:
  - În stânga: componente conectate (modularizare la nivel macro).

    - În dreapta: detaliul unui modul intern (subdivizat intern pe responsabilități).
- Ideea este că fiecare modul trebuie să fie:
    - clar definit,
    - coerent intern,
    - ușor de înțeles chiar și detașat de restul.

4. Continuity



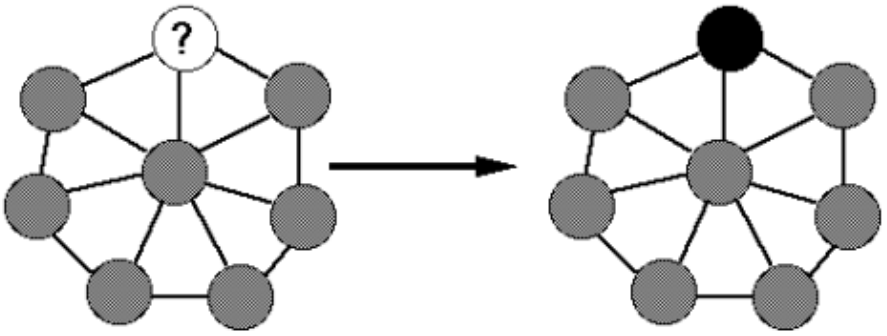
Cand ai de facut modificari mici, sa nu trebuiasca decat sa faci modificari in putine locuri

```
➢ Un sistem are continuitate? Pot sa modific intr-un loc fara sa trebuiasca
sa modific in tot sistemul? Va afecta o componenta sau toate componentele?
Continuitatea se refera doar la schimbari mici? Nu. Se refera la orice
schimbare. -> exista o proportionalitate intre cerintele de schimbare si
impactul schimbarii in cod -> adica daca avem niste cerinte mici, vom avea
de facut si schimbari mici? Sau daca avem cerinte medii, pot sa deduc ca vor
fi afectate cateva module, dar nu ft multe? Si doar daca
cerintele/schimbarile sunt extraordinar de mari, doar atunci va fi impactat
tot sistemul? La intrebarile astea ne gandim cand vine vorba de continuitate
     o Daca vreau sa vad ca o functie e continua, la functia aia am un
argument x -> f(x) -> ma astept ca daca valorile lui x sunt apropiate pe axa
Ox (care e ordonata) , atunci si valorile lui f(x) sa fie apropiate pe axa
Oy -> asta inseamna continuitate
     o La continuitate -> trebuie sa existe o proportionalitate intre
schimbarea cerintelor, amplitudinea schimbarii codului si amplitudinea
schimbarii cerintelor
     o Contraexemplu : avem constante : val 6 in 100 de locuri de program ->
```

```
vreau sa modific pe 6 in 7 -> daca 6 are mai multe semnificatii in program?
De ex 6 repr intr-o parte a codului luna a 6-a dintr-un an, in loc repr a 6-
a zi din sapt , in alt loc repr nr 6 de pe un zar samd. -> vreau sa schimb a
6-a zi cu a 7-a zi -> din cauza ca nu am folosit constante simbolice, ci am
lucrat cu valori hardcodate -> te costa ft mult pt ca trebuie sa revizitezi
toate cele 100 de instante in care am folosit 6
     o Exemplu : constantele simbolice
```

5. Protection

- Effects of an abnormal run-time condition is confined to a few modules
  - Example: Validating input at source
  - Counter-example: Undisciplined exceptions



Daca un modul e gresit/ e atacat, atunci nu se va propaga eroarea foarte mult (ideal ar fi sa ramana eroarea in cateva module maxim)

```
➢ Ce presupune protectia? Porneste de la ideea de izolare, dar nu izolare
de impactul schimbarii, ci izolare de impactul erorii -> daca se defecteaza
un modul, va crapa tot sistemul sau va afecta doar un nr mic de alte module?
    Ex: un vapor are mai multe cabine, ele sunt compartimentate -> daca se
sparge un compartiment, apa nu se va putea duce in toate celelalte
compartimente -> nu se va scufunda vaporul -> regula e ca compartimentele sa
fie ft bine delimitate -> etans delimitate -> ca o eroare sa nu poata afecta
tot sistemul
    o E ultimul criteriu de modularitate
    o Exemplu : Cea mai buna metoda de protectie e sa verificam inainte de a
incepe executia ca inputul este cel bun -> adica nu pornim executia decat
daca verificam ca inputul dat este acceptabil de modulul respectiv -> de ex,
daca primesc un nr negativ nu execut pt ca e posibil sa crape ceva ->
raportez undeva ca am lucrat pe un input eronat -> cand validez inputul
inainte de excutie introduc un factor de protective ca sa evit un rezultat
```

```
eronat
    o Contraexemplu : mecanismul de exceptii -> mai exact, o tratare
nedisciplinata a exceptiilor -> de ex, las toate exceptiile sa urce pana in
main, nu conteaza sau nu prind niciun fel de exceptii sau le tratez aiurea -
> risc sa se propage o exceptie/eroare dintr-un loc in care a aparut catre
nivelele superioare care pot fi in cu totul alte module -> nu vrem asta ,
acest lucru tine de protectie
```

Meyer's Five Rules of Modularity

- Direct Mapping
  - consistent relation between problem model and solution structure
- Few Interfaces
  - Every component should communicate with as few others as possible
- Small Interfaces
  - If any two components communicate at all, they should exchange as little information as possible
- Explicit Interfaces
  - Whenever two components A and B communicate, this must be obvious from the text of A or B or both
- Information Hiding

1. Direct Mapping

- Keep the structure of the solution compatible with the structure of the modeled problem domain
  - clear mapping (correspondence) between the two

Impact on:

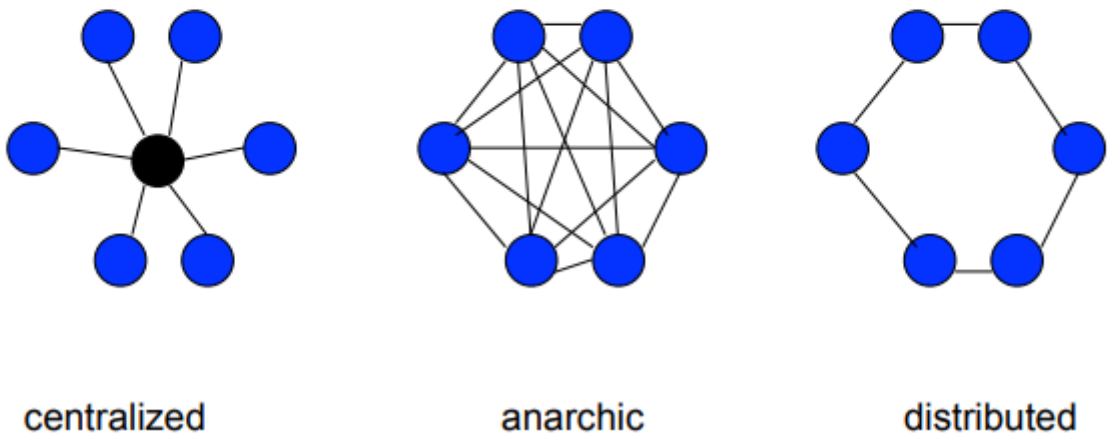
- Continuity
  - easier to assess and limit the impact of change
- Decomposability
  - decomposition in the problem domain model as a good starting point for the decomposition of the software

```
Trebuie sa te asiguri ca avem o mapare simpla, directa intre structura
solutiei si structura problemei modelate
```

```
▪ Avantaje : impact asupra : continuitate (o schimbare afecteaza putin
celelalte module) -> concepte unitar definite , decompozabilitate
```

2. Few Interfaces

- Every module should communicate with as few others as possible
  - rather n-1 than n(n-1)/ 2
  - Continuity, Protection, Understandability, Composability



La primul, eroarea de la cel din mijloc poate afecta toate celelalte module. Ceva de genul si la al doilea
Al treilea, fiind distribuit -> nu avem multe legaturi -> este ok

```
Cu cat am mai putine legaturi intre module, cu atat imi va fi mai usor cand
voi avea de schimbat ceva, nu va impacta grav tot sistemul -> un modul sa
comunice cu cat mai putine alte module, dar nici cu 0 module
```

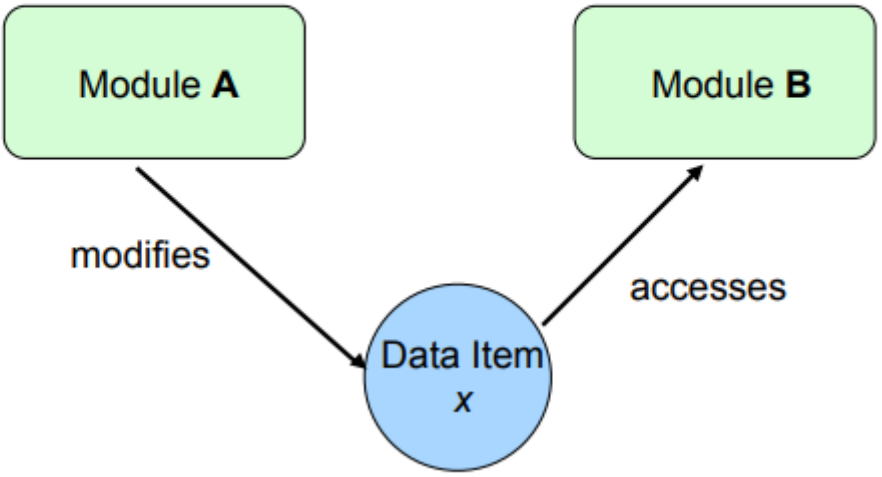
3. Small Interfaces

- If two modules communicate, they should exchange as little information as possible
  - limited "bandwidth" of communication
  - Continuity and Protection

Latimea de banda de comunicare trebuie sa fie mica

4. Explicit Interfaces

- Whenever two modules A and B communicate, this must be obvious from the text of A or B or both.
  - Decomposability and Composability
  - Continuity, Understandability
- The issue of indirect coupling
  - data sharing




```
2 module comunica explicit atunci cand dependenta lor e vizibila din
descrierea ambelor module -> ma pot uita la un modul si pot sa vad cu cine
comunica si ca acele relatii de comunicare sunt toate canalele de comunicare
ale acelui modul cu restul lumii -> cum ar putea fi altfel? Avem date
partajate : 2 module nu comunica direct, ci prin intermediul unor date
globale la care au acces mai multe module si ele le pot modifica sau accesa
-> cand comunicarea nu e explicita e o prob de intelegere -> comunicare
implicita : ce am scris mai sus cu faptul ca ele comunica prin date
```

Rule 2 + Rule 3 + Rule 4 Rephrased

- Few Interfaces: *"Don't talk to many!"*
- Small Interfaces: *"Don't talk a lot!"*
- Explicit Interfaces: *"Talk loud and in public! Don't whisper!"*

o Putine interfete : sa nu comunici cu multe module, ci cu cat mai putine
o Interfete mici : sa faci comunicarea asta sa fie cat mai limitata : se refera atat la nr de canale de comunicare, cat si la nr de date schimbate
o Interfete explicite : rel de comunicare sa fie publice si directe, explicite

5. Information Hiding

Motivation: design decisions that are subject to change should be hidden behind abstract interfaces, i.e. components

- Components should communicate only through well-defined interfaces
- Each component is specified by as little information as possible

- **Continuity:** If internal details change, client components should be minimally affected
  - not even recompiling or linking

```
Nu se refera doar la specificatorii de acces, ci si la module care expun
interfete -> informez celalte module ce pot sa apelez din modulul respectiv
prin intermediul interfetei sale : punctul cheie e echilibrul intre ce e
public si ce e ascuns : principiul eisberg-ului -> mecanismele sa ascunda
cat mai mult si sa expuna cat mai putin -> daca am mecanismul si 90% din
metodele mele sunt publice -> nu vorbim despre ascunderea informatiei
```

🔶 Motivation:

- Deciziile de design care s-ar putea schimba în viitor trebuie ascunse în spatele interfețelor abstracte.
- Componentele trebuie să comunice doar prin interfețe bine definite.

Exemplu simplu:

- Schimbi algoritmul de sortare intern, dar interfața `sort(lista)` rămâne neschimbată ➔ restul aplicației nu e afectată.

🔶 Principiu important: "Specify by as little information as possible"

- Cu cât interfața expune mai puține detalii, cu atât este mai stabilă.
- Nu expune variabile interne, tipuri de date interne, implementări detaliate.
- Oferă doar semnătura metodei și contractul de funcționare.

🔶 Continuity:

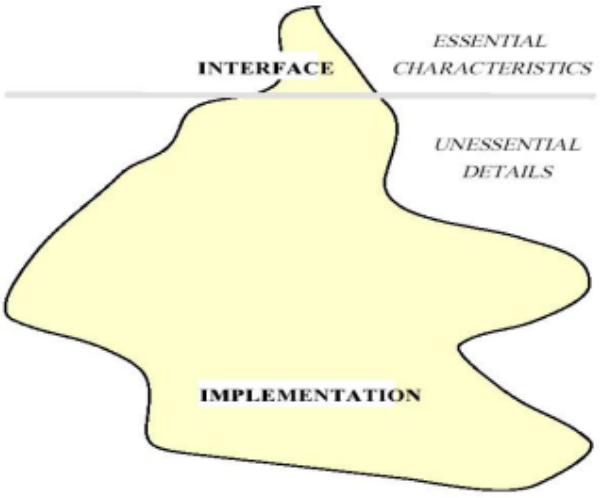
> Dacă detaliile interne se schimbă, clienții nu ar trebui să fie afectați.

- Ideal, restul sistemului:
  - nu trebuie recompilat;
  - nu trebuie redesenat;
  - nu trebuie rescris cod.

Exemplu:
Ai o aplicație care folosește o librărie de criptare `Encrypt()`.

- Dacă dezvoltatorul librăriei schimbă algoritmul de criptare intern (ex: AES în loc de DES), tu nu trebuie să modifici codul tău.

Abstraction vs. Information Hiding



Information hiding is one means to enhance abstraction!

Information hiding = ascunderea detaliilor interne
Abstraction = prezentarea doar a caracteristicilor esențiale

Ce reprezintă desenul:

- **INTERFACE** – partea vizibilă pentru restul sistemului:
  - expune doar ceea ce este necesar altor componente.
  - oferă o *abstracție* — adică oferă o imagine simplificată a ceea ce știe să facă componenta.

- **IMPLEMENTATION** – partea ascunsă:
  - detaliile interne de implementare (cod, algoritmi, structuri de date).
  - ceilalți nu trebuie să știe cum funcționează intern, doar cum să o folosească.
- **ESSENTIAL CHARACTERISTICS** – ce este important să știe restul sistemului.
- **UNESSENTIAL DETAILS** – detalii interne irelevante pentru exterior.

🔵 De ce este important Information Hiding?

- Permite **modificarea internă** a implementării fără a afecta alte componente.
- Reduce dependențele între module.
- Simplifică înțelegerea sistemului (tu vezi doar interfața, nu și complexitatea internă).

Exemplu concret:

- O bibliotecă care oferă:
  - `sort(lista)`
- Nu știi dacă folosește QuickSort, MergeSort sau alt algoritm. Atât timp cât respectă interfața, nu te interesează implementarea.

Software Architecture

Fowler's Ironic Definition of Architecture...

I define architecture as a word we use
when we want to talk about design
but want to puff it up to make it sound important

M.Fowler – "Who Needs an Architect", 2003

Definition of Software Architecture

"Software Architecture involves the description of

- **elements** from which systems are built,
- **interactions** among those elements,
- **patterns** that guide their composition and
- **constraints** on these patterns“

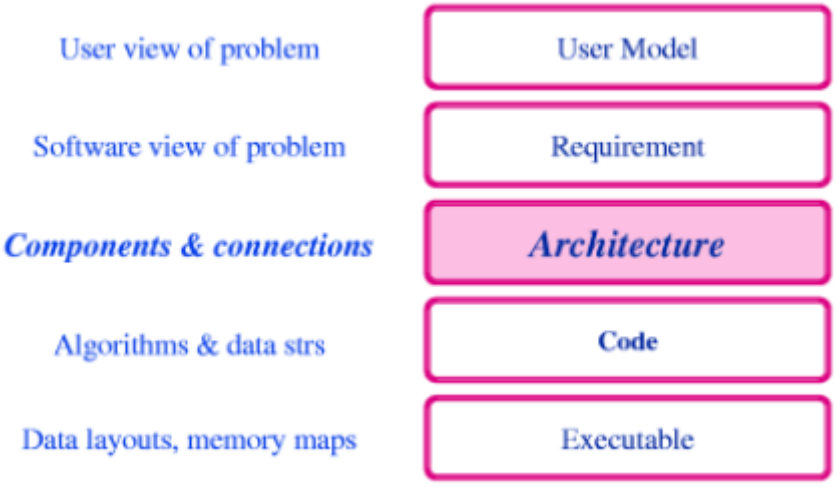
Shaw, Garlan – “Software Architecture”, 1996

“The software architecture of a program or computing system is the **structure** (or structures) **of the system**, which comprise

- **software components**,
- the externally **visible properties** of those components and
- the **relationships** among them"

Bass,Clements, Kazman – “Software Architecture in Practice”, 1998

The place of Software Architecture




```
Exact ca la sistemele modulare -> entitati cat mai autonome care
interactioneaza intre ele cat mai simplu si coerent (interfetele intre
entitati si relatiile dintre ele)
La nivelul cel mai exterior sistemului : partea de interactiune al userului
cu sistemul (USER MODEL) : introduc datele etc.
Apoi avem nivelul de requirement : cerintele (orientat spre ce face
sistemul)
Urm nivel : arhitectura
Urm nivel : codul
Urm nivel : executabilul
User model + requirements : ce e sistemul ? -> Codul + executabilul : ne
```

```
spun cum e sistemul, cum e el implementat -> arhitectura e intre cele 2
categorii -> arhitectul repr puntea dintre cele 2 maluri : cum e sistemul si
ce face sistemul -> trebuie sa le armonizeze : intelege cerintele
clientului, se asigura si ca ce construieste e posibil dpdv tehnic
Arhitectura e despre lucrurile care sunt greu de schimbat!!! -> schimbare si
agilitate
```

More Definitions

- In most successful software projects, the expert developers working on that project have a shared understanding of the system design. This shared understanding is called 'architecture.' This understanding includes how the system is divided into components and how the components interact through interfaces. These components are usually composed of smaller components, but the architecture only includes the components and interfaces that are understood by all the developers. **[R.Johnson, 2003]**
- Architecture is about the important stuff. Whatever that is. **[R.Johnson, 2003]**
- Architecture is the decisions that you wish you could get right early in a project, but that you are not necessarily more likely to get them right than any other. **[R.Johnson, 2003]**
- ...things that people perceive as hard to change **[M.Fowler, 2003]**

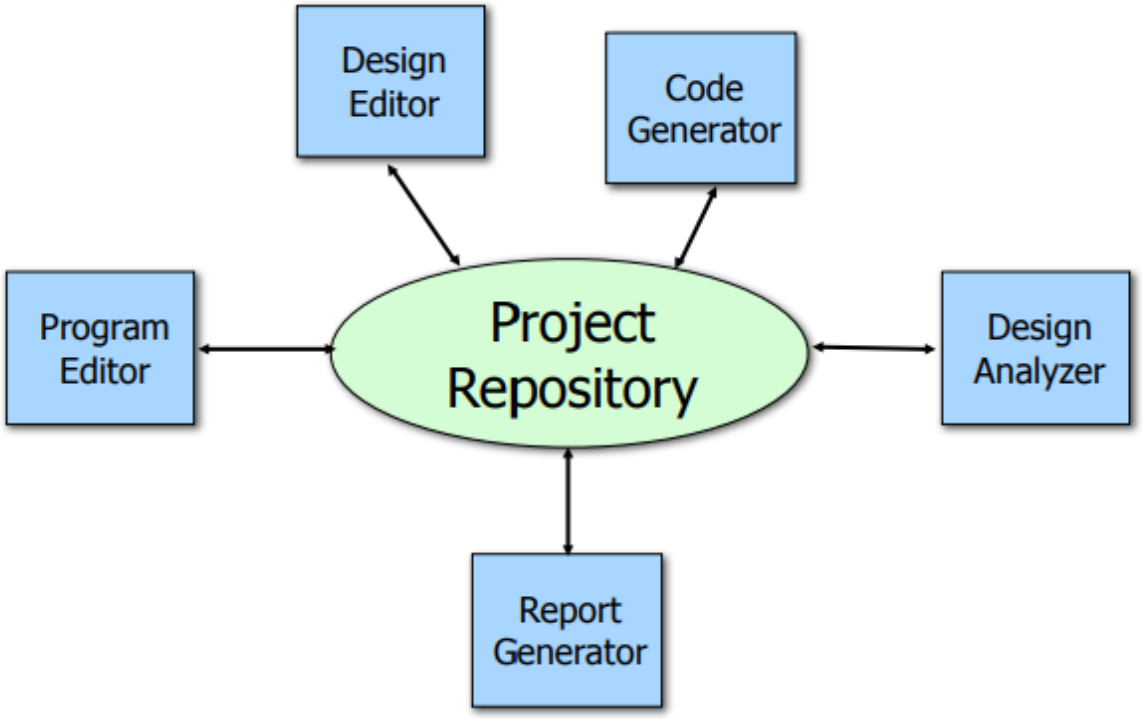
Non-Functional Requirements

- **Performance**
  - Small number of subsystems ➔ large-grained components
  - Reduce communication
- **Security**
  - Layered structure
  - Most critical layer in the inside
  - High-level of security validation
- **Maintainability**
  - Easy to change ➔ Fine-grained, self-contained components
  - Producers of data separated from consumers
  - Avoid shared data structures

```
o Ex : la providerul de net, de ex digi, spune : viteza pe care ti-o
garantez e x (cerinta functionala)
    ▪ Daca spun ca nu poate cadea conexiunea mai mult de 5 min pe an ->
cerinta nefunctionala
o Avem cerinte de performanta , de securitate si de mentenanta
o Performanta :
    ▪ Nr mic de subsisteme -> componente cu granulatie mare
    ▪ Reducerea comunicarii -> comunicare rapida
    ▪ Performanta de timp -> n-am voie sa am latenta -> nu adoptam un stil
layered
o Securitatea :
    ▪ Structura stratificata
    ▪ Cel mai critic strat in interior -> nucleu
    ▪ Validarea securitatii la nivel inalt
    ▪ Protectie impotriva erorilor
o Mentenanta : intretinerea unui sistem : cat ma costa daca vreau sa adaug
un nou feature
    ▪ Sistemul trebuie sa fie usor de modificat
    ▪ Stilurile arhitecturale sunt legate de cerintele functionale
    ▪ Cod scris in asamblare -> greu de scris si de mentinut
```

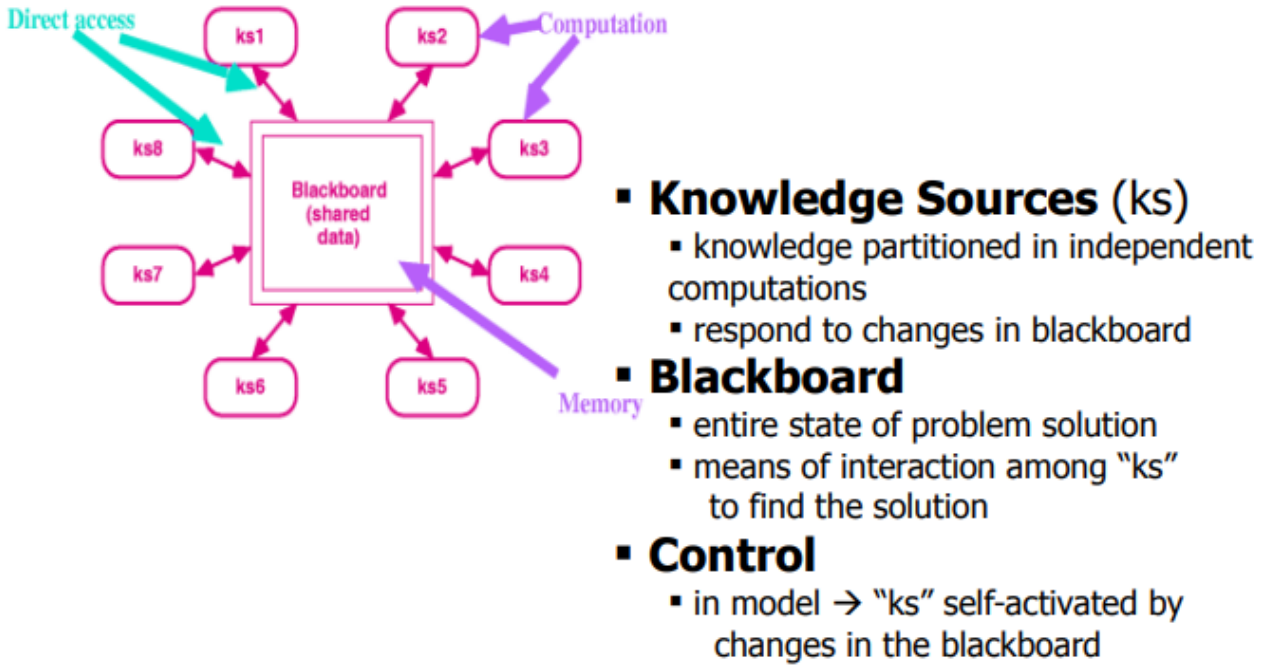
Some Architectural Styles...

Repositories. An Example



toate dreptunghiurile albastre -> knowledge sources

Repositories - Blackboard



Avem in centrul repository-ul, care e la fel ca o baza de date -> contine toate datele de care au nevoie elementele componente -> componentele s. n. knowlegde sources -> sunt niste entitati care care consuma date din acest repository, pot scrie, pot actualiza date etc.
o Blackboard vine de la faptul ca n-ai o solutie imediata, mai multi oameni colaboreaza, poate fi ca un puzzle complex si mai multi oameni lucreaza la el -> toata lumea se uita la aceeasi tabla
o Concluzie : avem mai multe creiere care toate interactioneaza cu datele din repo -> pot sa aceeseze repository-ul in care se afla date fie scriind, citind etc.
o Producatorii si consumatorii de date sunt total independenti -> subsistemele nu trebuie sa-si faca griji de responsabilitati auxiliare, cum ar fi backup-ul, securitatea etc.
o Se poate adauga oricand un nou modul, un nou knowlegde source, care poate sa inceapa sa consume si sa produca date in acel model -> modulele raman independente intre ele -> tot ce trebuie sa stie e cum sa se conecteze la acel repository

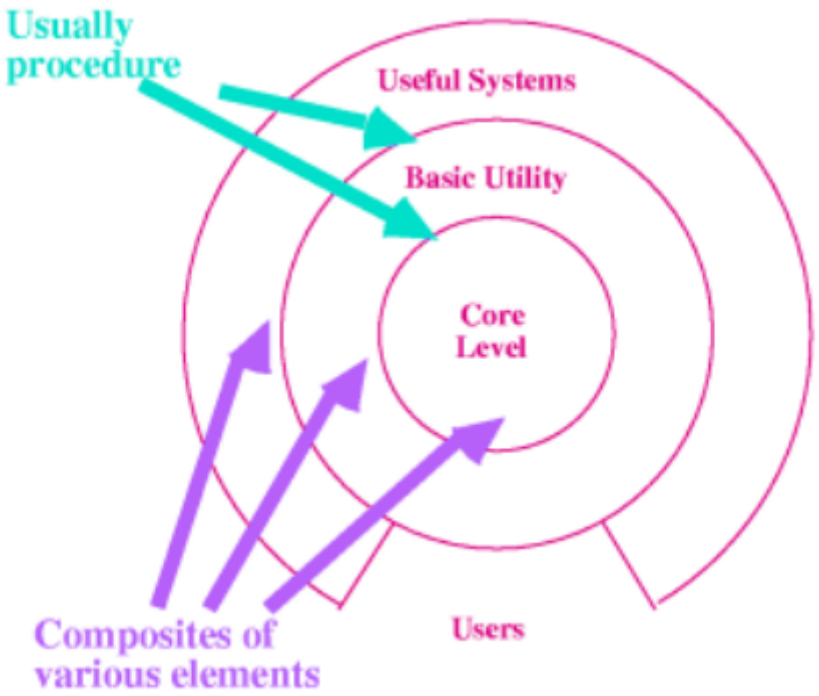
Repositories – Summary

- ✓ Efficient way to share large amounts of data
- ✓ Data producers and consumers are totally independent
- ✓ Subsystems don't have to care about auxiliary responsibilities
  - e.g. backup, security, recovery from error

- ✗ The common data-model is a compromise among subsystems
- ✗ Expensive translations to a new model
- ✗ Repository forces a centralized policy on all subsystems
  - ✗ e.g. backup, security, access control

```
o Avantaje : oricand am de gestionat date, exista un overhead -> nu mai e
necesar sa-l rezolvam pt ca fiecare modul foloseste modelul de date comun
    ▪ E un mod eficient de a share-ui cantitati mari de date intre diverse
subsisteme/componente/entitati
    ▪ Producatorii si consumatorii de date sunt independenti
    ▪ Subsistemele nu au responsabilitati auxiliare (ex: sa faca backup)
o Dezavantaje :
    ▪ modelul cu date comune e un compromis printre subsisteme
    ▪ translatarile catre un nou model sunt costisitoare
    ▪ repository-ul forteaza o politica centralizata pentru toate
subsistemele
```

Layered Architecture (a.k.a. Abstract Machine)



E ca o ceapa(R. Marinescu, 2025) -> adica are mai multe straturi si ca sa ajungi la mijloc, trebuie sa treci prin toate

```
Arhitectura in 3 straturi : avem ideea de concentric : ca sa ajung la miez, in centru, trebuie sa trec prin toate straturile intermediare
o Fiecare strat are un nivel de comunicare ft limitat
o Stratul exterior : va putea fi accesat din afara sistemului prin UI -> nu foloseste decat stratul imediat inferior(mai din interior) -> apoi acest strat imediat inferior ofera o interfata catre stratul asta exterior si acceseaza informatii din stratul imediat inferior si tot asa -> de ex, stratul din exterior “Useful Systems” nu poate sa aceeseze core-ul (centru acestor straturi)
o Orice cerere care vrea sa ajunga in core level trebuie neaparat sa treaca prin toate straturile o Fiecare strat are ft putin legaturi : ofera o interfata pt nivelul de deasupra si acceseaza interfata nivelului de dedesubt
o Se poate lua un strat si se poate reutiliza pt alt sistem pe care il construiesc
```

Summary

- ✓ Changeable and portable architecture
- ✓ Suitable for incremental development
  - Provide services as soon as a layer is implemented
- ✓ Support reuse

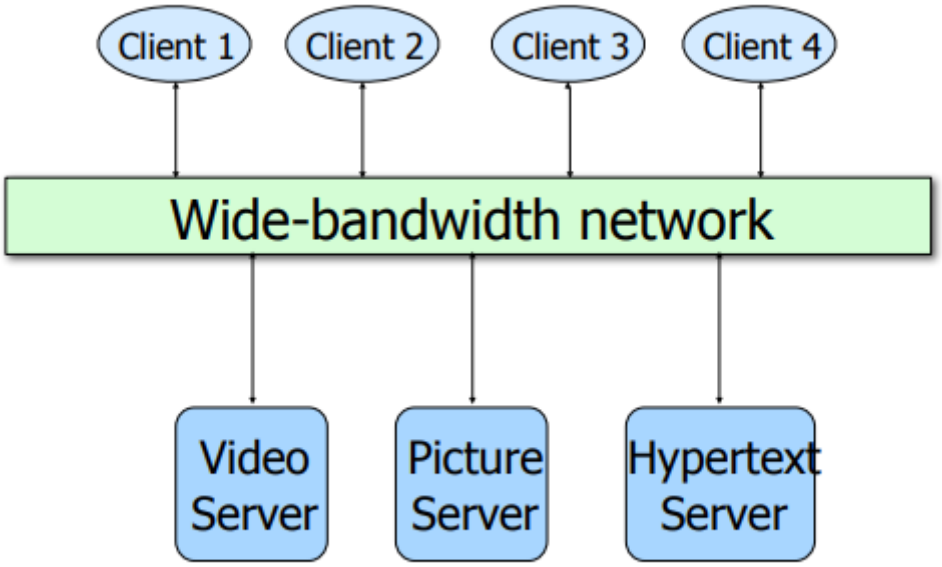
```
o Avantaj mare :
    ▪ securitatea : fiecare strat ar putea avea avea un mecanism de
protectie/securitate prin drepturile de acces
    ▪ e o arhitectura portabila si schimbabila
    ▪ se poate reutiliza un strat deja folosit pt un alt sistem
    ▪ e avantajos pt dezvoltarea incrementala
```

In caz de ceva, nu trebuie sa schimbi tot, schimbi doar layer-ul pe care vrei sa il schimbi

- ✗ Hard to achieve such a rigorous structuring
  - ✗ Hard to find the proper levels of abstraction
- ✗ Reduces performance by increased communication

```
o Dezavantaje :
    ▪ E greu de atins o structura atat de riguroasa
    ▪ Reduce performanta prin cresterea comunicarii
```

Client-Server. An Example



o Niste module care produc niste servicii -> fiecare are propriul modul de date complet independente intre ele
o Am o gramada de module client care consuma servicii de la servere
o Clientii sunt complet decuplati intre ei
o Fiecare server prezinta o interfata publica -> clientii acceseaza serviciile din aceasta interfata

Summary

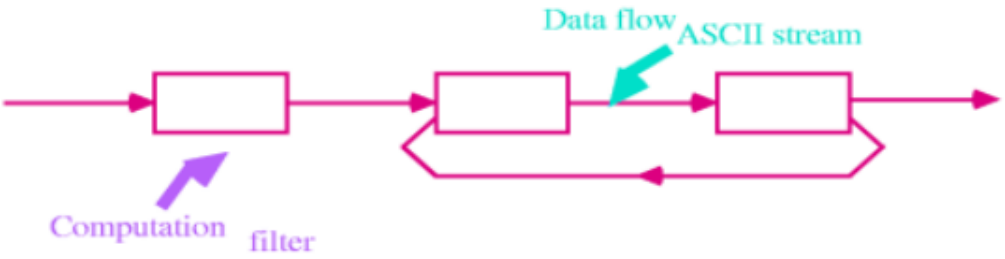
- ✓ Architecture is distributed
  - ✓ Easy to add and integrate new servers
  - ✓ System can be reconfigured dynamically
- ✓ Servers are not aware of clients
  - ✓ Neither identity nor number
- ✓ Each server can organize its own data-model
  - ✓ better then the centralized data-model in *Repository*

- ✖ Performance problems if large amounts of data are exchanged
- ✖ Hard to anticipate problems with integrating data from a new server

```
o Avantaje:
    ▪ Arhitectura e distribuita : e ft usor sa adaugi noi clienti si noi
servere -> orice sistem se poate reconfigura dinamic (Cand zic client ma
```

```
refer la un alt modul, nu la oameni) -> E ft usor pt clienti sa se
mute/conecteze pe alt server -> serverele nu au habar de clienti, sunt
decuplate , serverul e o entitate de sine statatoare
    ▪ Ideea de compozabilitate e ft puternica : microservicii -> fiecare
serviciu e complet independent si are propriul model de date si poate avea
propriul limbaj de programare -> din microservicii compunem un serviciu mai
mare o
Dezavantaje :
    ▪ Apar probleme de performanta( comunicare prin retea) cand apare un
trafic mare de date -> fiecare server are propriul modul de date , iar un
client are foarte rar nevoie de date de la un singur server
    ▪ E greu sa anticipam probleme cand integram noi date de la un nou
server
```

Pipes and Filters



- **Filter**
  - incrementally transform some flow of data at inputs to data at outputs
  - share no state with other filters
  - don't depend on the upstream and downstream filters
- **Pipe**
  - Move data from a filter output to a filter input
  - form graphs of data transmission

Prin pipe circula datele intr-o directie
Filtrele sunt fiecare independente

```
o Unde am nevoie? Avem ideea de compozabilitate : modulele s. n. filtre si
au 2 capete : 1 intrare si 1 iesire -> la intrare primeste niste date -> si
la iesire avem nsite date procesate -> avem nuclee de procesare -> fiecare
nucleu are un sg task de calcul si transmite catre output un rezultat ->
trebuie sa inlantuim mai multe nuclee de procesare
 Construim un sistem din mai multe comportamente simple si individuale
(sistemul if this then that)
```

Este un model foarte des folosit atunci când ai un flux de date care trebuie procesat pas cu pas.

FILTER

- Un filtru primește date la intrare ➔ aplică o **transformare** ➔ emite datele la ieșire.
- Este **incremental** — procesează datele bucățică cu bucățică.
- Foarte important:
  - **Nu are stare partajată** cu alte filtre.
  - **Nu depinde** de cine e înainte sau după el (upstream/downstream).
- Este autonom — știe doar să transforme datele pe care le primește.

Exemplu concret:

Să zicem că procesăm un text:

Filter	Ce face?
Filtru 1	Convertire la majuscule
Filtru 2	Eliminare spații în exces
Filtru 3	Extrage doar primele 100 caractere

- Fiecare filtru știe doar să-și facă propria transformare, nu știe de restul.

PIPE

- Pipe-ul face **legătura** între filtre.
- Mută datele de la ieșirea unui filtru la intrarea următorului.
- Practic formează **graful de transmitere a datelor**.
- Permite conectarea mai multor filtre în serie, dar și în rețea (pot exista ramificații).

Avantaje	Dezavantaje
Foarte modular	Nu se pot rezolva ușor probleme cu feedback (înapoiere de date)
Fiecare filtru e independent	Timp total de procesare poate fi mare (mai ales dacă datele sunt mari)
Foarte ușor de extins	

Exemplu real

- Procesare de fișiere log
- Procesare streamuri video/audio (compresie, filtrare, codare)

- Compilatoare (fiecare etapă: lexing, parsing, semantic analysis, code generation)
- În shell-ul Unix: `cat file.txt | grep "abc" | sort | uniq`

Summary

- ✓ Understand behavior as a composition of the behavior of individual filters
- ✓ Support for filter reuse
- ✓ Systems are easy to maintain and enhance
  - ✓ By changing or adding new filters

```
o Avantaje :
    ▪ Utilizat in sisteme pentru tranzactii cu carduri
    ▪ Pot compune si pot altera comportamente ft usor, e ft usor sa adaug
sau sa imbunatatesc un comportament
    ▪ Filtrele pot fi reutilizate ft bine, comportamentele pot fi practic
customizate usor
```

- ✖ Not good for interactive applications
- ✖ Filters need a common format for data transfer
- ✖ Each filter must parse and un-parse the data stream → overhead

```
o Dezavantaje :
    ▪ ca sa pot conecta 2 noduri, ele trebuie sa vb aceeasi limba ->
outputul de la un filtru nu poate sa fie acelasi cu inputul de la altul
garantat -> nu pot presupune cu 2 filtre au acelasi format de date ->
fiecare filtru trebuie sa despacheteze ce primeste si apoi sa impacheteze la
loc -> overhead de performanta si de implementare
    ▪ nu e buna pt aplicatiile interactive
    ▪ filtrele au nevoie de un format comun pt datele ce le transfera
    ▪ avem overhead : fiecare filtru trebuie sa despacheteze ce primeste si
apoi sa impacheteze la loc
```